

Final Project 实验报告

王思宇

2026 年 6 月 28 日

目录

1	Final Project 实验报告	2
1.1	引言	2
1.1.1	工程结构	3
1.1.2	编译流水线与输出命名	5
1.2	词法分析与语法分析	6
1.2.1	语法边界	6
1.3	类型检查	7
1.4	AST 到 IR+ 的翻译	8
1.4.1	运行期语义与检查	9
1.4.2	数组和对象布局细节	9
1.5	Quad 表示、Tiles 与 Def/Use	10
1.6	SSA 转换与 Phi 函数	11
1.7	SSA Quad 优化	11
1.7.1	Constant Propagation	12
1.7.2	Loop Invariant Hoisting	12
1.7.3	Induction Variables & Strength Reduction	13
1.8	SSA Quad 到 ARM 指令选择	13
1.9	寄存器分配	14
1.10	ARM 汇编代码生成	14
1.11	测试、构建与用户手册	15
1.11.1	构建	15
1.11.2	编译测试目录	15
1.11.3	运行测试目录	16
1.11.4	编译和运行单个文件	16
1.11.5	解释器 oracle	17

1.11.6 回归测试	18
1.11.7 Fuzz 测试	19
1.11.8 优化性能测试	20
1.11.9 运行时回归测试	21
1.12 实现中遇到的问题	22
1.12.1 空 label / 空 block	22
1.12.2 Fallthrough 边上的 Phi Copy	22
1.12.3 可选运行期语义	22
1.12.4 后端立即数范围	23
1.12.5 函数 prologue 位置	23
1.12.6 HW6-HW10 优化组合	23
1.13 结论	23
1.14 参考资料	24

1 Final Project 实验报告

1.1 引言

本报告说明 final/ 中 FMJ 编译器的完整实现，旨在解释本项目代码如何把一个 .fmj 源程序一步步变成可在 qemu-arm 上运行的 ARM 汇编。

最终编译器入口是 final/tools/fmjcc/main.cc，解释器 oracle 是 final/tools/fmjinterp↔
↔ /main.cc。编译流水线如下：

```

1 FMJ source
2   -> flex/bison parser
3   -> AST + semantic table
4   -> IR+
5   -> canonical IR+
6   -> Quad + basic blocks + def/use
7   -> SSA Quad
8   -> SSA optimizations
9   -> ARM-like selected instructions
10  -> scheduling
11  -> liveness + interference graph
12  -> register allocation
13  -> final ARM assembly

```

从理论上讲，编译器通常被拆成前端、中端和后端。前端负责把文本程序变成有类型的结构化程序表示，并拒绝不符合语言定义的输入；中端把语言相关结构逐步降低到更适合分析和优化的中间表示；后端再把中间表示映射到目标机器的寄存器、指令和调用约定。这样分层的意义是让每一层维护较小的抽象边界：parser 不需要知道 ARM，寄存器分配也不需要知道 FMJ 语法，只需要消费前一阶段保证好的不变量。

代码包根目录为 `final/`。默认的 `make compile` 和 `make run*` 会递归扫描 `test/` 下所有 `.fmj`。其中 `test/all` 是从 HW、PARSING 和老师发布的 `FINALREPORTtests` 中按裸文件名去重得到的主测试集；`test/submit/report_example.fmj` 是本报告使用的贯穿示例。

本报告使用 `final/test/submit/report_example.fmj` 作为贯穿例子：

```
1 public int main() {
2     int[] a = {1, 2, 3, 4};
3     class Acc acc;
4     int n;
5     acc = new Acc();
6     n = length(a);
7     putint(acc.sum(a, n));
8     putch(10);
9     return 0;
10 }
11
12 public class Acc {
13     int bias;
14
15     public int sum(int[] a, int n) {
16         int i;
17         int s;
18         i = 0;
19         s = this.bias;
20         while (i < n) {
21             if (a[i] > 0) {
22                 s = s + a[i];
23             } else {
24                 s = s + 0;
25             }
26             i = i + 1;
27         }
28         return s;
29     }
30 }
```

它覆盖了 `main`、类、字段、方法、数组字面量、对象创建、动态方法调用、`length`、`while`、`if`、数组下标和外部输出，足以展示各阶段的主要数据结构。

1.1.1 工程结构

`final/` 保留各次作业中 `include/`、`lib/`、`tools/` 的总体组织方式，把原先分散在 HW2-HW12 中的组件统一放入同一个 CMake 工程：

- `final/tools/fmjcc/main.cc`：完整编译器入口，负责串接 `parser`、`semantic`、`IR`、`Quad`、`SSA`、`优化`、`指令选择`、`调度`、`寄存器分配`和最终汇编输出。
- `final/tools/fmjinterp/main.cc`：解释器和语义检查 `oracle`。它不参与最终代码生成，但用于确认 `reject` 和端到端运行结果。

- `final/lib/frontend/` 与 `final/include/frontend/`: 从 PARSING/ 集成进来的 flex/bison parser 源码。
- `final/lib/ast/` 与 `final/include/ast/`: AST 节点、NameMap、语义分析。
- `final/lib/ir/` 与 `final/include/ir/`: AST 到 Tree IR+、canonicalization、临时变量和 label 管理。
- `final/lib/quad/` 与 `final/include/quad/`: Tree IR 到 Quad、basic block、SSA。
- `final/lib/quadflow/`: Quad 级控制流和数据流信息。
- `final/lib/opt/` 与 `final/include/opt/`: SCCP、LICM、归纳变量识别和强度削弱。
- `final/lib/instr/` 与 `final/include/instr/`: ARM 指令选择、advDFG、pre-schedule、schedule、汇编程序表示。
- `final/lib/reg/` 与 `final/include/reg/`: 干涉图、图着色寄存器分配、spill 改写。
- `final/lib/util/`: XML 读写、AST/IR/Quad 打印、source printer。
- `final/scripts/collect_tests.sh`: 从各 HW、PARSING/test 和 FINALREPORTtests 收集 .fmj 测试，并按裸文件名去重。
- `final/scripts/test_expect.sh`: 读取测试文件开头的 EXPECT: PASS/FAIL 标记，供回归脚本判断编译期失败是否符合预期。
- `final/scripts/compile_submit.sh`: 实现 make compile 的批量编译和多优化模式输出。
- `final/scripts/run_submit.sh`: 实现 make run* 的链接和 qemu 执行。
- `final/scripts/compile_regression.sh`: 检查编译器接受/拒绝输入，以及是否出现空 label/block。
- `final/scripts/interpreter_regression.sh`: 用解释器对照编译运行结果。
- `final/scripts/runtime_regression.sh`: 快速检查 HW10/HW12 中运行端到端样例。
- `final/scripts/fuzz_regression.sh`: 对含输入程序做解释器 oracle fuzz。
- `final/scripts/all_mode_regression.sh`: 对所有测试比较六种优化模式的返回码和输出是否一致。
- `final/scripts/opt_benchmark.sh`: 对老师新增的长运行样例测量六种优化模式下的 qemu 运行时间。

这样组织代码，每个阶段仍能对应到原作业模块，报告中提到的阶段可以直接回到相应 lib/ 或 include/ 目录查看代码。fmjcc/main.cc 则只承担流水线胶水层职责，不把各阶段算法混在入口文件里。

工程结构背后的基本原则是阶段接口比阶段内部更重要。每个阶段除了生成下一个阶段的输入，还要保证若干不变量。例如语义分析之后每个表达式都应当能查到类型，Quad 之后每条语句都应当有 def/use，SSA 之后每个版本化 temp 应当只有一个定义，寄存器分配之后最终汇编不应再出现未着色虚拟寄存器。Final 的目录划分正是围绕这些接口组织的。

1.1.2 编译流水线与输出命名

fmjcc 的主流程写在 `main()` 中，核心顺序如下：

1. `runParser()` 调用 `fdmjParser()` 把 `.fmj` 解析成 AST，并输出 `.1.fmj` 与 `.2.ast`。
2. `xml2ast()` 读回 XML AST，恢复 C++ AST 对象。
3. `semant_analyze()` 构建 `Name_Maps` 与 `AST_Semant_Map`，输出 `.2-semant.ast`。
4. `ast2tree()` 翻译为 Tree IR+，输出 `.3.irp`。
5. `canon()` 规范化 IR+，输出 `.3-canon.irp`。
6. `tree2quad()` 铺砖生成 Quad，输出 `.4.quad` 和 `.4-xml.quad`。
7. `blocking()` 划分 basic block，控制流和数据流分析输出 `.4-quadwithflow-xml.quad`。
8. `quad2ssa()` 转 SSA，输出 `.4-ssa.quad`。
9. 按 `--opt-mode` 运行 SCCP、LICM、IV/SR，最终输出 `.4-ssa-final-<mode>.quad`。
10. `buildAdvDFGprog()`、`buildPreScheduleProg()`、`runInstructionSelectionPass()` 和 `scheduleProg()` 生成虚拟寄存器 ARM 指令，输出 `.5-xml.asm` 与中间 `.s`。
11. `preDataFlowPass()`、`buildIgProg()`、`coloring()`、`asmprog2colored()` 做寄存器分配和 spill 改写，输出 `.colored.s` 与最终 `.<mode>.s`。

`make compile` 会对 `test/` 下递归找到的所有 `.fmj` 在六个 `mode` 中分别执行这条流水线，因此同一个源文件可以在 `output/none/`、`output/const/`、`output/loop1/`、`output/loop2/`、`output/allloop/`、`output/allopt/` 下看到不同优化设置的完整产物。语法或语义错误的程序不会中断整个批量编译，而会记录在对应 `mode` 的 `compile-results.txt` 和 `compile-failures.txt` 中。贯穿例子 `report_example.fmj` 在 `allopt` 模式下会生成：

```
1 report_example.1.fmj
2 report_example.2-semant.ast
3 report_example.3.irp
4 report_example.4.quad
5 report_example.4-ssa.quad
6 report_example.4-ssa-final-allopt.quad
7 report_example.allopt.s
```

此外还会输出 XML、flow、blocked Quad、colored assembly、compile/link/run log 等辅助文件，便于定位任意阶段的问题。

这些中间文件也体现了编译器调试的基本方法：不要把“源程序到目标汇编”看成一个黑盒，而是把每个语义保持的转换都落盘。如果某个程序最终输出错误，可以先检查 `.2-semant.ast` 的类型信息是否正确，再检查 `.3.irp` 是否保留了源程序语义，继续检查 `.4.quad` 的 `def/use`、`.4-ssa.quad` 的 `phi`、优化后 Quad 和最终汇编。这样错误可以被定位到第一个破坏语义不变量的阶段。

1.2 词法分析与语法分析

词法分析和语法分析解决的是“文本如何变成结构”的问题。词法分析把字符流切成 token，理论模型通常是正则语言和有限自动机；语法分析把 token 流按上下文无关文法组织成语法树，理论模型是 CFG 和 LR/LALR 自动机。使用 flex/bison 的好处是我们把 token 规则和 grammar 规则显式写出来，由工具生成自动机，避免手写递归下降时遗漏优先级、结合性或错误恢复细节。

前端代码来自 PARSING/，已经以源代码形式集成到 final/lib/frontend/lexer.ll 和 final/lib/frontend/parser.yy，而不是调用外部 parser binary。final/CMakeLists.txt 使用 FLEX_TARGET 和 BISON_TARGET 在构建时生成 lexer.cc、parser.cc，并把它们链接进 fmjcc 和 fmjinterp，所以编译器和解释器共享同一套语法边界。

词法分析器定义在 lexer.ll。它识别关键字、标识符、非负整数、运算符和标点，同时跳过空白、// 行注释和 /* ... */ 块注释。无法识别的字符会打印 Illegal input 并使 parser 失败。

语法分析器定义在 parser.yy。每个产生式直接构造 FDMJAST.hh 中的 AST 节点。例如：

- MAINMETHOD → public int main() { VARDECLLIST STMLIST } 构造 MainMethod。
- TYPE → class ID | int | int[] | int[NUMBER] 构造 Type。
- VARDECL → TYPE ID ; | TYPE ID = { NUMBERLIST } ; 构造 VarDecl。
- EXP → EXP + EXP 等二元表达式构造 BinaryOp, EXP '[' EXP ']' 构造 ArrayExp, EXP '.' → ID '(' EXPLIST ')' 构造 CallExp。

对贯穿例子，parser 会构造一个 Program，其 main 指向 MainMethod，cd1 中有一个 ClassDecl (Acc)。int[] a = {1,2,3,4}; 变成一个 VarDecl，其 TypeKind 为 ARRAY，初始化值是 vector<IntExp*>; acc.sum(a, n) 变成 CallExp(obj=IdExp("acc"), name=IdExp("sum"), par=[IdExp("a"), IdExp("n")])。

fmjcc 的 parser 阶段在 runParser() 中完成。parser 成功后：

- source_printer.cc 从 AST 重新打印规范化源码到 .1.fmj，格式和注释不再保留。
- ast2xml(root, nullptr, true, false) 输出原始 AST 到 .2.ast。

如果 bison 解析失败，ASTParser::error() 会报告出错位置和详细消息，fdmjParser() 返回 nullptr，fmjcc 以非零状态拒绝该输入。

1.2.1 语法边界

Final 按 HW2/docs/FDMJ2026Specification.md 确认语法边界。一个重要边界是变量声明：

```
1 VarDecl → Type id [= ArrayInit] ;
2 ArrayInit → { CONST (, CONST)* }
```

也就是说,变量声明只允许没有初始化,或者使用数组字面量初始化;`int i = 0;`并不是合法 FMJ 语法。这个边界影响了几个早期 HW2 测试文件,解释器和编译器都会按 `specification` 拒绝它们。

另一个边界是继承。`Specification` 只要求单层继承,本实现也只接受单层继承,并在语义分析中拒绝多层继承和循环继承。这样对象布局、方法查找和类型兼容规则都能保持和课程 `specification` 一致。

1.3 类型检查

类型检查属于静态语义分析。语法树只说明程序“长得像合法程序”,但不说明 `a[i]` 中 `a` 是否真的是数组、`acc.sum(a, n)` 是否有这个方法、参数类型是否匹配。类型系统的作用是在运行前排除一类无意义或危险的程序,并给后续翻译阶段提供确定的信息。对面向对象语言来说,类型检查还要处理类名空间、继承、字段查找、方法签名和赋值兼容性。

类型检查入口是 `final/lib/ast/semantanalyzer.cc` 中的 `semant_analyze()`。它先调用 `makeNameMaps`↔
↔ () 构建全局名字表,再用 `AST_Semant_Visitor` 遍历 AST 并给表达式和语句节点填入语义信息。

名字表 `Name_Maps` 定义在 `include/ast/namemaps.hh`, 主要保存:

- 类名集合、父类关系。
- 每个类的字段列表和方法列表。
- 每个方法的形参、局部变量和返回类型。
- `main` 方法的局部变量表。

语义表 `AST_Semant_Map` 定义在 `include/ast/semant.hh`, 把 AST 节点映射到 `Semant` 信息,包括表达式类型、类型参数和 `lvalue` 信息。`fmjcc` 会把带 `NameMap` 和 `Semant` 的 AST 输出为 `.2-semant.ast`。

本实现检查的主要规则包括:

- 类名、字段名、方法名和变量引用必须存在。
- `int`、`int[]`、`class C` 的赋值和返回类型必须兼容。
- 子类对象可赋给父类变量;方法重写需要检查参数和返回类型。
- `if`、`while` 条件需要是 `int` 语义下的布尔表达式。
- 数组下标对象必须是数组,下标必须是 `int`。
- 方法调用的实参数量和类型必须匹配。
- `specification` 中只允许单层继承,本实现拒绝多层继承和循环继承。

在贯穿例子中,`acc` 的类型为 `class Acc`, `acc.sum(a, n)` 的解析依赖 `Name_Maps` 中 `Acc.sum` 的方法签名:第一个显式实参 `a` 必须是 `int[]`,第二个实参 `n` 必须是 `int`,返回类型是 `int`。`this.bias` 会通过当前访问类 `Acc` 找到字段 `bias`,并标记为 `int` 类型的可取地址表达式。

类型错误通过 `cerr` 打印具体位置和错误原因，然后终止当前编译；回归脚本使用解释器的 `--check` 模式确认被拒绝的 `.fmj` 文件确实不符合语法或语义规则。

这里使用 `NameMap` 加 `AST-to-Semant` 映射，是符号表技术的一种直接实现。符号表把源程序中的名字绑定到声明，语义表把 `AST` 节点绑定到推导出的类型。理论上，这相当于在 `AST` 上执行一遍属性计算：声明节点产生环境，表达式节点从环境读取信息并合成类型，语句节点检查控制流和赋值约束是否满足语言规则。

1.4 AST 到 IR+ 的翻译

`IR` 是源语言和目标机器之间的桥梁。`AST` 保留了很多源语言结构，例如对象、方法调用、数组字面量和 `while`；目标机器只认识寄存器、内存、跳转和调用。`AST` 到 `IR+` 的理论任务是降低抽象层次：把高级语法结构翻译成更接近机器模型、但仍然不依赖具体寄存器和具体指令编码的树形中间语言。

`AST` 到 `IR+` 的翻译入口是 `final/lib/ir/ast2tree.cc` 中的 `ast2tree()`，主 visitor 是 `ASTToTreeVisitor`。输出类型定义在 `include/ir/treep.hh`，包括 `Seq`、`Move`、`Cjump`、`Jump`、`LabelStm`、`Binop`、`Mem`、`Call`、`ExtCall`、`Eseq` 等节点。`fmjcc` 把这一阶段结果写到 `.3.irp`，再经过 `canon()` 输出 `.3-canon.irp`。

翻译时构建了两个重要表：

- `Method_var_table`：为方法的形参、局部变量和返回值分配 `IR temp`，并记录每个 `temp` 的 `tree::Type`。
- `Class_table`：构造统一对象表示 `UOR`，为字段和方法指针分配对象内偏移。

本项目的运行时布局如下：

- `int[]` 数组首 `word` 保存长度，元素从第二个 `word` 开始，因此 `a[i]` 翻译为 `Mem[a + (i + 1) * 4]`。
- 对象中先放字段槽，再放方法指针槽；`new Acc()` 会调用 `malloc`，把字段初始化为 0，并写入 `Acc^sum` 等方法入口地址。
- 对象方法调用 `obj.m(args...)` 被翻译成从对象方法槽取函数指针，并把 `obj` 作为第一个实参传入。
- 函数或 `main` 到末尾没有显式 `return` 时补默认 `return 0`。

对贯穿例子：

- `int[] a = {1,2,3,4}` 生成一段 `malloc` 和若干 `Move(Mem(...), Const(...))`，首 `word` 写入长度 4。
- `acc = new Acc()` 生成对象分配、字段 `bias` 写 0、方法槽写 `Name("Acc^sum")`。
- `s = this.bias` 生成 `Mem[this + bias_offset]`。
- `while (i < n)` 生成条件 `label`、`body label` 和 `done label`，并用 `Cjump("<", i, n, body, done)` 表示分支。

1.4.1 运行期语义与检查

运行期语义是静态类型检查无法完全表达的约束。比如数组下标是否越界、引用是否为空、除数是否为 0，通常要等程序执行到具体路径和具体输入时才知道。编译器可以选择插入检查来把这类错误转化为确定的运行期行为，也可以在语言未规定时把它们视为未定义行为；Final 也明确区分默认语义和额外检查语义。

Final 默认只保留 **specification** 或原 **lab** 代码已经需要的运行期表示规则：

- 数组布局使用首 **word** 记录长度，数组字面量和 `new int[n]` 使用同一布局。
- 对象创建时初始化祖先类字段、本类字段和方法指针；字段槽位先写 0，符合 **specification** 中 **class field** 初值为 0 的例子。
- 数组下标越界检查默认开启。
- 局部 **class** 引用声明后写 0。
- 函数或 **main** 执行到末尾没有显式 **return** 时默认返回 0。

另额外补了一些运行期语义检查，但默认不开启：局部 **int** 默认 0、局部数组引用默认空、空对象/空数组检查、负长度数组检查、除零检查。显式使用 `--runtime-checks` 或别名 `-↔↔ -extra-runtime-semantics` 时才开启这些语义检查；运行期错误通过 `exit(-1)` 结束，因此 **shell** 观察到的退出码为 255。`--no-runtime-checks` 显式保持默认行为。

这里区分默认行为和额外语义，是因为 **specification** 并没有完整规定所有未定义行为。默认编译器只承诺定义良好程序的结果正确；解释器在 **regression** 中会标记依赖额外语义的样例，默认跳过这些输出等价比较，开启 `RUNTIME_CHECKS=1` 后再要求完全一致。

1.4.2 数组和对象布局细节

数据布局是把语言级对象映射到线性内存的规则。只要确定了数组和对象中每个字段的偏移，后续 **IR**、**Quad** 和 **ARM** 阶段就不需要再理解”数组”或”类”的高级概念，只需要做地址计算和内存访问。这个阶段的正确性要求是：同一种语言值在所有构造点和使用点必须采用同一布局。

数组字面量和 `new int[n]` 使用同一布局是 **Final** 集成时特别修正的一点。数组指针指向长度 **word**，元素地址是：

```
1 base + (index + 1) * address_length
```

这样 `length(a)` 可以直接翻译成 `Mem[a]`，`putarray(n, a)` 和 `getarray(a)` 也可以和 `libsysy` 侧约定对齐。贯穿例子中的 `a[i]` 会先检查 `i >= 0` 和 `i < Mem[a]`，再访问 `Mem[a + (i + 1) * 4]`。

对象布局采用统一对象表示 **UOR**。**Class_table** 先为所有字段分配偏移，再为所有方法名分配方法指针偏移。对象创建时：

1. 调用 `malloc(total_size)`。
2. 沿祖先到当前类的顺序初始化字段槽。

3. 对数组字段字面量分配数组并把指针写入字段。
4. 对每个方法槽写入实际实现类的函数 label, 例如 `Acc^sum`。

动态调用 `acc.sum(a, n)` 因此不直接跳到静态 label, 而是从对象中取 `sum` 的方法槽, 再通过 `blx` 间接调用。这样子类重写方法时, 只要对象方法槽中写入子类实现, 调用点不需要改动。

1.5 Quad 表示、Tiles 与 Def/Use

Quad 是一种接近三地址码的中间表示。树形 IR 适合表达嵌套表达式, 但不方便做数据流分析, 因为一个树节点里可能同时包含多个计算步骤。Quad 把复杂表达式拆成一条条简单语句, 每条语句至多定义少量目标并使用若干源操作数。这样, 控制流图、活跃变量分析、SSA 和寄存器分配都可以围绕”语句的 def/use 集合”来定义。

IR+ 到 Quad 的翻译入口是 `final/lib/quad/tree2quad.cc` 中的 `tree2quad()`。Tree2Quad $\leftrightarrow$ visitor 对 canonical IR 做铺砖, 把树形表达式变成接近三地址码的 Quad 指令。输出包括 `.4.quad` 和 XML 形式 `.4-xml.quad`, 随后 `blocking()` 与控制流分析输出 `basic block` 和 `.4- $\leftrightarrow$  quadwithflow-xml.quad`。

本项目使用的主要 tile 如下:

IR+ 形态	Quad 指令	def	use
<code>Move(Temp t, e)</code>	<code>QuadMove(t, e)</code>	<code>t</code>	<code>e</code> 中的 temp
<code>Move(Mem addr, e)</code>	<code>QuadStore(e, addr)</code>	空	<code>e</code> 和 <code>addr</code> 中的 temp
<code>Move(Temp t, Binop $\leftrightarrow$ (op,l,r))</code>	<code>QuadMoveBinop(t, $\leftrightarrow$ op, l, r)</code>	<code>t</code>	<code>l</code> 、 <code>r</code> 中的 temp
<code>Move(Temp t, Call $\leftrightarrow$ (...))</code>	<code>QuadMoveCall(t, $\leftrightarrow$ call)</code>	<code>t</code>	调用目标和实参 temp
<code>Move(Temp t, $\leftrightarrow$ ExtCall(...))</code>	<code>QuadMoveExtCall(t, $\leftrightarrow$ extcall)</code>	<code>t</code>	外部调用实参 temp
<code>Cjump(op,l,r,t,f)</code>	<code>QuadCJump</code>	空	<code>l</code> 、 <code>r</code> 中的 temp
<code>Jump L</code>	<code>QuadJump</code>	空	空
<code>Label L</code>	<code>QuadLabel</code>	空	空
<code>Return e</code>	<code>QuadReturn</code>	空	<code>e</code> 中的 temp
<code>Mem(base+offset) 地址计算</code>	<code>QuadPtrCalc + QuadLoad/Store</code>	地址 temp 或 load 目标	<code>base</code> 、 <code>offset</code> 、 <code>store</code> 源

`tree2quad.cc` 中的 `addTermUse()` 负责从 `QuadTerm` 中抽取 temp 放入 use 集; 每种 Quad 构造时显式创建 def 和 use 集。这些集合会被后续 `dataflow`、`SSA`、`liveness` 和寄存器分配复用。

对贯穿例子的 `s = s + a[i]`, 翻译后大致会形成:

```

1 t_idx_addr <- ptr_calc(a, (i + 1) * 4)
2 t_val <- load(t_idx_addr)
3 s <- s + t_val

```

其中 `ptr_calc` use `a` 和 `i`, `load` def `t_val` use `t_idx_addr`, 加法 def `s` use `s` 和 `t_val`。

1.6 SSA 转换与 Phi 函数

SSA 的核心思想是每个变量版本只被定义一次。这样很多优化和分析会更简单：如果某个 `temp` 版本是常量，它不会在后面被重新赋值推翻；如果一个 `use` 引用某个版本，就能直接找到唯一 `def`。控制流 `join` 处的多来源值用 `phi` 函数表示。`phi` 不是机器指令，而是编译器中间表示中的“根据前驱 `block` 选择对应值”的抽象。

SSA 转换入口是 `final/lib/quad/quadssa.cc` 中的 `quad2ssa()`。在进入 SSA 前，`blocking()` 先把 Quad 程序切成 `basic blocks`, `dataFlowProg()` 和 `ControlFlowInfo::computeEverything()` 建出前驱、后继、支配树、支配边界和 `liveness` 信息。

SSA 实现分三步：

1. `placePhi()` 收集每个变量的定义 `block`，对其定义集合计算 `iterated dominance frontier`。为了避免插入过多 `phi`，本实现使用 `pruned SSA`：只有当该变量在候选 `join block live-in` 时才插入 `phi`。
2. `renameVariables()` 沿支配树递归重命名变量。每个原 `temp` 有一个版本栈，遇到 `use` 时读栈顶版本，遇到 `def` 时创建新版本并入栈。
3. 遍历当前 `block` 的后继，按 `predecessor label` 更新后继中 `phi` 参数；最后 `cleanupUnusedPhi` ↔ `()` 删除未使用 `phi`。

在贯穿例子的 `while` 中，`i` 和 `s` 都在循环前定义，也在循环体内重新定义。循环头有来自入口路径和回边路径的两个 `predecessor`，因此 SSA 中会插入类似：

```

1 i_2 = phi(i_0 from preheader, i_1 from loop_body)
2 s_2 = phi(s_0 from preheader, s_1 from loop_body)

```

之后循环条件使用 `i_2`，循环体中的累加使用 `s_2`，体尾 `i = i + 1` 会产生新版本并作为回边上的 `phi` 参数。

1.7 SSA Quad 优化

优化的目标是在保持可观察行为不变的前提下改写程序，使其更快、更短或更适合后端生成代码。SSA 适合做优化，是因为定义和使用关系更清晰，数据流信息可以沿着版本化变量传播。Final 的三个优化分别覆盖常量信息传播、循环中不变计算外提、以及循环归纳变量的代数化简。

优化由 `fmjcc --opt-mode MODE` 控制，`final/Makefile` 的 `run targets` 与模式一一对应：

- none: 不做优化, 用于 make run。
- const: 运行 Constant Propagation, 用于 make run-const。
- loop1: 运行 Loop Invariant Hoisting, 用于 make run-loop1。
- loop2: 运行 Induction Variable & Strength Reduction, 用于 make run-loop2。
- allloop: 运行两个循环优化, 用于 make run-allloop。
- allopt: 运行全部优化, 用于 make run-allopt, 也是单文件编译默认模式。

fmjcc/main.cc 将每个 pass 拆成 runSccpPass()、runLicmPass() 和 runIvPass(), 并在每个 pass 后刷新 last_label_num 和 last_temp_num, 避免新 label/temp 与旧编号冲突。最终优化结果输出为 .4-ssa-final-<mode>.quad。

1.7.1 Constant Propagation

常量传播可以看成是一个数据流问题: 每个 SSA temp 的值在格上取 unknown、具体常量或 multiple values。算法不断根据语句语义更新格值, 直到到达不动点。到达不动点后, 如果某个 use 的值是具体常量, 就可以把这个 use 替换成常量; 如果某个条件恒真或恒假, 还可以进一步简化控制流。

SCCP 常量传播实现位于 final/lib/opt/opt.cc, 入口是 optProg()。它在 SSA Quad 上传播常量格值, 并把已知常量替换进后续 use。由于 SSA 中每个版本只定义一次, 传播时不需要处理同一个变量被多次覆盖的问题。

在贯穿例子中, `else { s = s + 0; }` 的右操作数 0 是常量, const 或 allopt 模式会尽量把这个常量直接保留到更靠后的 Quad/ARM 立即数形式中; 如果某些条件可由常量判定, 也会裁剪对应分支。

1.7.2 Loop Invariant Hoisting

循环不变代码外提基于一个简单观察: 如果循环体中某个表达式每次迭代都算出同一个值, 并且把它提前到循环前不会改变异常、内存和定义使用关系, 那么它没有必要在每次迭代中重复计算。理论上需要依赖循环识别、支配关系和 def/use 信息, 确保外提后的定义支配循环内所有使用点, 且不会越过可能改变其操作数的语句。

LICM 的代码位于 final/lib/opt/loopheaderwithflow.cc 和 final/lib/opt/loophoistfunc↔.cc。findLoopHeadersWithFlow() 用控制流信息识别回边和循环头, loopHoistFunc() 分析循环内语句的 use/def, 找出所有操作数在循环外定义或在循环中不改变的表达式, 并把它们提到循环前的 preheader。

在贯穿例子的循环中, n 来自循环外, this.bias 在循环前读入 s, 循环条件 `i < n` 中的 n 是 loop-invariant。对于更复杂的地址或算术表达式, 如果其操作数都不被循环体重定义, LICM 会把它们从 loop body 移到 loop preheader。

1.7.3 Induction Variables & Strength Reduction

归纳变量优化利用循环中的线性变化模式。若 i 每次迭代增加一个常量，那么 $i * 4$ $\leftrightarrow$ $\hookrightarrow$ 、 $base + i * 4$ 等表达式也是随迭代线性变化的派生归纳变量。强度削弱的理论基础是把昂贵运算替换成等价的便宜运算，例如把循环中的乘法替换为每轮加一个固定步长。

归纳变量优化位于 `final/lib/opt/loopinduction*.cc` 和 `final/lib/opt/loopstrengthreduction` $\leftrightarrow$ $\hookrightarrow$.cc，入口是 `loopInductionStrengthReductionPass()` 和 `loopInductionCleanupPass()`。它先识别 basic induction variable，例如贯穿例子中的 $i = i + 1$ ；再识别由它派生出的表达式，例如数组地址计算里的 $(i + 1) * 4$ ；最后用递增更新替代每次循环中的乘法。

对 $a[i]$ ，未优化时每次迭代都要计算 $(i + 1) * 4$ 。strength reduction 后可以维护一个随 i 同步递增的地址或偏移 temp，把乘法变成加法。cleanup pass 会删除不再需要的派生变量计算。

1.8 SSA Quad 到 ARM 指令选择

指令选择把机器无关的中间表示映射到目标 ISA。理论上常用“树模式匹配”或“tiling”：每个目标指令能覆盖 IR/Quad 中某种形态的计算，例如 `add dst, src, #imm` 可以同时覆盖加法和小立即数。优秀的指令选择会尽量选择覆盖更多计算、代价更低、且符合目标机器立即数和寻址限制的 tile。

指令选择相关代码在 `final/lib/instr/`。 `buildAdvDFGprog()` 和 `buildPreScheduleProg()` $\leftrightarrow$ $\hookrightarrow$ 先把每个 Quad block 变成适合调度的图和预调度结构，`runInstructionSelectionPass()` 调用 `selectInstructionsForBlock()` 为每条 Quad 选择 ARM 指令模板，结果写入 `.5-xml.asm` 和中间 `.s`。

重要选择规则包括：

- 常量通过 `mov`、`movw`、`movt` 物化；小立即数直接用于 `add/sub`。
- `QuadMoveBinop` 选择 `add`、`sub`、`mul`、`sdiv`。
- `QuadPtrCalc` 如果 `offset` 在 ARM `load/store` 立即数范围内，会与后续 `load/store` 折叠成 `[base, #offset]`。
- `QuadLoad` 选择 `ldr`，`QuadStore` 选择 `str`。
- 外部调用如 `putint`、`malloc` 选择 `bl function`，返回值从 `r0` 移回虚拟 temp。
- 对象方法调用先把对象方法槽中取出的函数指针物化，再使用 `blx ip` 间接调用；显式参数和 `this` 按调用约定放入 `r0` 起始的实参寄存器。

函数入口、返回和分支由调度阶段保留在 block 边界处。入口 `label`、`prologue`、`body`、`epilogue` 的顺序在 `schedule.cc` 和后续汇编生成中固定，避免循环回边跳回函数入口时重复执行压栈。

对贯穿例子的 `acc.sum(a, n)`，IR 阶段已经把调用目标翻译为 `Mem[acc + sum_method_offset` $\leftrightarrow$ $\hookrightarrow$]，指令选择会生成读取函数指针、准备 `this/a/n` 参数、`blx` 间接跳转、再把 `r0` 结果保存到虚拟 temp 的指令序列。

1.9 寄存器分配

寄存器分配解决的是虚拟寄存器到有限物理寄存器的映射问题。理论上，它通常被建模为图着色：如果两个值在同一程序点同时 **live**，它们不能放在同一个寄存器，于是在干涉图中连一条边；给图中每个节点分配颜色，就是给每个虚拟寄存器分配物理寄存器。物理寄存器不够时，需要 **spill** 到栈上。

寄存器分配输入是已经选择和调度后的虚拟寄存器汇编。主要代码位于 `final/lib/reg/` 和 `final/lib/instr/asmdatflow.cc`。

第一步是 `preDataFlowPass()`。它把 `call` 指令对 `r0` 到 `r3` 的破坏加入 `def` 集，这样跨调用仍然 **live** 的虚拟 `temp` 会与 `caller-saved register` 干涉。

第二步是 `buildIgProg()`。它先对每个函数运行汇编级 **liveness analysis**，再构造 **interference graph**：

- 每个 `use`、`def`、`liveout temp` 都成为图节点。
- 对每个 `def` 和每个 `liveout temp` 加干涉边。
- 对 `move` 指令保留 `move pair`，并在源和目标之间不立即加边，为 **coalescing** 留机会。

第三步是图着色，入口是 `coloring()`。实现使用 **simplify**、**coalesce**、**freeze**、**spill**、**select** 的循环：

- `simplify()` 删除度数小于 `k` 的非机器寄存器节点。
- `coalesce()` 用 **George** 条件安全合并 `move` 相关节点。
- `freeze()` 在无法合并时冻结低度 `move`。
- `spill()` 选择高度数节点作为 **potential spill**。
- `select()` 反向弹栈分配颜色；无可用颜色则标记为 **spilled**。

机器寄存器 `temp` 是 **precolored node**。默认 `k=9`，即可分配 `r0` 到 `r8`；项目保留 `r9/r10` 等寄存器用于 **spill** 代码生成。

贯穿例子中，循环里的 `i`、`s`、数组基址和方法参数在不同程序点 **live**。**liveness** 会让同一时刻活跃的值互相干涉，图着色保证它们不会被分到同一物理寄存器。

1.10 ARM 汇编代码生成

最终汇编生成需要把前面阶段的抽象都落实到目标平台 **ABI** 上。**ABI** 规定参数和返回值使用哪些寄存器、调用者/被调用者保存哪些寄存器、栈如何对齐、函数入口和返回如何维护 `sp/fp/lr`。如果寄存器分配只给出了“这个 `temp` 使用 `r3`”，汇编生成还要保证函数 **prologue/epilogue**、**spill** 栈槽、外部调用和全局 `label` 都符合 **ARM** 工具链和 **qemu** 的期望。

最终汇编生成由 `asmprog2colored()` 完成。它读取每个函数的 **Coloring**：

- 未 **spill** 的虚拟 `temp` 被替换成对应的 **ARM register**。

- 机器寄存器 `temp` 直接替换成固定寄存器，例如 `r0`、`sp`、`lr`。
- `spilled temp` 分配函数内栈槽，使用前插入 `ldr`，定义后插入 `str`。
- 根据 `spill` 栈槽数量调整函数栈帧大小。

`fmjcc` 会把着色后汇编写入 `.colored.s`，同时按当前模式写出最终 `.s`，例如 `report_example`↔
↔ `.allopt.s`。 `make run*` 使用如下命令用 `qemu` 跑这个 `.s`：

```
1 arm-linux-gnueabi-gcc -mcpu=cortex-a72 --static program.s libsys32.s -lm
2 qemu-arm program.arm
```

最终 ARM 文件包含全局函数标签、函数 `prologue/epilogue`、`label`、`branch`、`load/store`、算术指令和对 `libsysy` 的外部调用。贯穿例子运行时会输出：

```
1 10
```

其中 `10` 来自 `Acc.sum({1,2,3,4}, 4)`。

1.11 测试、构建与用户手册

测试部分对应编译器正确性的工程方法。单个样例只能证明某条路径没有暴露错误，不能证明编译器整体可靠；因此 **Final** 同时使用编译回归、解释器差分测试、运行时回归和随机 `fuzz`。差分测试的理论思路是：如果两个独立执行机制对同一输入程序产生同一可观察结果，那么后端某个阶段破坏语义的概率会显著下降；`fuzz` 则通过大量随机输入覆盖人工样例难以枚举的路径。

1.11.1 构建

在 `final/` 下执行：

```
1 make build
```

该命令用 `CMake/Ninja` 构建：

- `build/fmjcc`：完整编译器。
- `build/fmjinterp`：解释器和语义检查 `oracle`。

1.11.2 编译测试目录

```
1 make compile
```

默认递归编译 `test/` 下所有 `.fmj`，并在 `output/<mode>/` 中为每个成功编译的源文件输出至少以下文件：

- `.1.fmj`：AST 重新打印的规范化 FMJ 源文件。
- `.2-semant.ast`：带 `NameMap` 和语义信息的 AST。

- .3.irp: IR+ 树。
- .4.quad: 从 IR+ 生成的 Quad 程序。
- .4-ssa.quad: SSA 转换后的 Quad。
- .4-ssa-final-<mode>.quad: 指定优化模式后的 Quad。
- .<mode>.s: 该模式最终 ARM 汇编。

make compile 会一次生成 none、const、loop1、loop2、allloop、allopt 六种模式的产物。无法通过 parser 或 semantic checker 的输入会写入 compile-results.txt 和 compile-failures.txt, 用于确认 reject 行为。

1.11.3 运行测试目录

```
1 make run          # 无优化
2 make run-const    # Constant Propagation
3 make run-loop1     # Loop Invariant Hoisting
4 make run-loop2     # Induction Variable & Strength Reduction
5 make run-allloop  # 两个循环优化
6 make run-allopt    # 全部优化
```

make run* 会先递归编译 test/ 下所有 .fmj, 再逐个链接并运行成功编译的程序。每个程序都会打印一行 mode=... result=... rc=... src=..., 非空 stdout/stderr 会跟随输出; 被前端拒绝的程序会以 COMPILE_FAIL 行列出, 如果程序运行超时 (含有死循环), 则会出现 RUN_FAIL, rc=124 (即 qemu 对应退出码)。

如果程序需要输入, 默认不自动填充 stdin, 而是让 qemu 直接连接当前终端; 因此从终端运行 make run* 时, 含有 getint、getch 或 getarray 的程序仍会在运行到对应语句时等待用户输入, 程序运行过程中的 stdout/stderr 也会即时显示。批量运行默认启用 RUN_TIMEOUT, 脚本使用 timeout --foreground 保证 timeout 包裹下 qemu 仍可从终端读取输入。非交互场景可以通过 INPUT 变量给每个程序提供同一份输入:

```
1 make run-allopt INPUT="1 2 3 4"
```

如果设置了 INPUT, Makefile 会把它传给 run_submit.sh, 脚本用 printf '%s\n' "\$INPUT" 喂给每次 qemu 运行, 并把 stdout/stderr 捕获到文件后按块打印; 如果没有设置, 则脚本不重定向 qemu 的 stdin/stdout/stderr, 输入和即时输出完全由正在运行的程序自己处理。

批量运行和回归测试中的 qemu timeout 默认是 2 秒, 可通过 RUN_TIMEOUT 调整:

```
1 make run-allopt RUN_TIMEOUT=5
2 make run-allopt RUN_TIMEOUT= # 关闭 timeout
```

1.11.4 编译和运行单个文件

```
1 build/fmjcc --k 9 --opt-mode allopt path/to/file.fmj
2 build/fmjinterp --check path/to/file.fmj
```



```
3 make run-one path/to/file.fmj
4 make run-one path/to/file.fmj OPT_MODE=none
5 make run-one path/to/file.fmj OPT_MODE=allopt
6 make run-one-all-mode path/to/file.fmj
```

`make run-one` 默认使用 `OPT_MODE=allopt`，也可以设置为 `none`、`const`、`loop1`、`loop2`、`allloop` 或 `allopt`。`make run-one-all-mode` 会对同一个文件依次运行六种优化模式，打印每个模式的 `process return code`、`stdout`、`stderr` 和 `harness` 打印出的 `FMJ return value`，并比较这些可观察结果是否完全一致。

`make run-one` 和 `make run-one-all-mode` 默认同样不自动提供 `stdin`，而是让 `qemu` 直连当前终端；脚本仍会捕获 `stderr`，因此 `harness` 打印的 `[fmj return]` 可以被解析并显示为 `fmj_return=...`。需要外部输入且希望自动比较多模式输出时，可以显式设置 `INPUT`；这样每个优化模式都会收到同一份输入，例如：

```
1 make run-one path/to/file.fmj INPUT="1 2 3"
2 make run-one-all-mode path/to/file.fmj INPUT="1 2 3"
```

这两个单文件目标也默认不对 `qemu` 设置 `timeout`；它们更适合人工运行和观察交互式程序。自动回归脚本会显式传入 `timeout`，避免批量测试被非终止程序卡住。

额外运行期语义可用：

```
1 build/fmjcc --runtime-checks path/to/file.fmj
2 make run-one path/to/file.fmj RUNTIME_CHECKS=1
```

1.11.5 解释器 oracle

`fmjinterp` 使用与编译器相同的 `parser` 和语义分析，保证“是否接受输入”的判断基于同一套语法和类型规则。解释器直接执行 `AST`，覆盖：

- `int`、数组、对象、继承和方法调用。
- `if`、`while`、`break`、`continue`、`return`。
- `getint`、`getch`、`getarray`、`putint`、`putch`、`putarray`。
- 32 位整数加减乘 `wraparound` 语义。
- 空引用、越界、负数组长度、除零等运行期错误。

解释器提供两种主要用途：

1. `--check` 只做 `parser` 和 `semantic check`，用于确认 `reject` 是否合理。
2. 普通执行模式输出 `stdout` 和退出码，用于和 `qemu` 运行的编译产物比较。

`fuzz` 时解释器还提供 `hash` 模式，和 `ARM harness` 使用相同的随机输入生成器以及相同的输出 `hash` 规则。这样大量随机输入不需要保存完整输出流，只比较最终 `hash` 和输入调用次数。

1.11.6 回归测试

`final/scripts/collect_tests.sh` 从 `HW`、`PARSING` 和 `FINALREPORTtests` 收集 `.fmj` 文件到 `test/all`。除老师要求的 `Make targets` 外，项目还保留：

```
1 make compile-regression
2 make interpreter-regression
3 make runtime-regression
4 make fuzz-regression
5 make all-mode-regression
6 make opt-benchmark
```

`collect_tests.sh` 会遍历整个仓库，排除 `final/` 自身，收集各 `HW`、`PARSING/test` 和老师新增测试中的 `.fmj` 文件到 `final/test/all`。去重规则按裸文件名执行：例如不同来源的 `bubblesort.fmj` 只保留一份，而不是保留 `HW12_...` 和 `PARSING_...` 两份。保留优先级为 `FINALREPORTtests` $\leftrightarrow$ `/newtests.fmj`、`FINALREPORTtests/oldtests.fmj`、其他 `FINALREPORTtests`、`HW12` 到 `HW1`、`PARSING`、其他来源。当前一共收集到 215 个唯一文件，其中 20 个带 `EXPECT: PASS`，33 个带 `EXPECT: FAIL`。

`EXPECT` 标记只影响测试 `oracle`，不影响编译器本身。`EXPECT: PASS` 表示 `parser` 和 `semantic checker` 必须接受该文件；`EXPECT: FAIL` 表示编译器必须拒绝，并且日志中要出现语法/语义诊断。没有标记的文件仍然通过解释器 `--check` 判断接受/拒绝是否一致。

`compile-regression` 只检查编译器是否稳定接受或拒绝输入，并扫描中间结果里是否出现空 `label/block`。对这 215 个输入，结果是：

```
1 total=215 pass=129 reject=86 expect_pass=20 expect_reject=33 unmarked_pass=109  $\leftrightarrow$ 
   $\hookrightarrow$  unmarked_reject=53 expect_mismatch=0 crash=0 timeout=0 empty_label_hits=0  $\leftrightarrow$ 
   $\hookrightarrow$  workdir=/tmp/final_compile_regression
```

这里 `reject=86` 再用解释器的 `--check` 模式确认。`interpreter-regression` 会分别运行解释器检查和编译器检查：

- 两者都拒绝，记为 `reject_match`。
- 只有编译器拒绝，记为 `compile_only_reject`。
- 只有解释器拒绝，记为 `interp_only_reject`。
- 两者都接受后，再比较解释器执行和 `qemu` 执行的 `stdout` 与退出状态。

最近一次完整验证结果如下：

```
1 make compile-regression
2 total=215 pass=129 reject=86 expect_pass=20 expect_reject=33 unmarked_pass=109  $\leftrightarrow$ 
   $\hookrightarrow$  unmarked_reject=53 expect_mismatch=0 crash=0 timeout=0 empty_label_hits=0
```

```
1 make interpreter-regression
2 total=215 run_match=110 reject_match=86 timeout_match=2 runtime_error_skip=13  $\leftrightarrow$ 
   $\hookrightarrow$  optional_semantics_skip=15 stress_runtime_skip=2 expect_pass_match=20  $\leftrightarrow$ 
   $\hookrightarrow$  expect_reject_match=33 mismatch=0 compile_only_reject=0 interp_only_reject=0  $\leftrightarrow$ 
   $\hookrightarrow$  link_fail=0 run_fail=0 timeout=2 runtime_checks=0
```

```

1  RUNTIME_CHECKS=1 make interpreter-regression
2  total=215 run_match=125 reject_match=86 timeout_match=2 runtime_error_skip=0 ↵
    ↵ optional_semantics_skip=0 stress_runtime_skip=2 expect_pass_match=20 ↵
    ↵ expect_reject_match=33 mismatch=0 compile_only_reject=0 interp_only_reject=0↵
    ↵ link_fail=0 run_fail=0 timeout=2 runtime_checks=1

```

```

1  make runtime-regression
2  total=24 pass=24 compile_fail=0 link_fail=0 run_fail=0

```

```

1  make fuzz-regression
2  fuzz_pass=46 fuzz_fail=0 fuzz_skip=3 iterations=10000

```

```

1  make all-mode-regression
2  total=221 run_consistent=118 reject_consistent=86 timeout_consistent=2 ↵
    ↵ optional_semantics_skip=15 expect_pass_consistent=20 ↵
    ↵ expect_reject_consistent=33 mismatch=0 interp_timeout=0 ↵
    ↵ unexpected_compile_reject=0 compiler_accept_invalid=0 runtime_checks=0

```

```

1  RUNTIME_CHECKS=1 make all-mode-regression
2  total=221 run_consistent=133 reject_consistent=86 timeout_consistent=2 ↵
    ↵ optional_semantics_skip=0 expect_pass_consistent=20 expect_reject_consistent↵
    ↵ =33 mismatch=0 interp_timeout=0 unexpected_compile_reject=0 ↵
    ↵ compiler_accept_invalid=0 runtime_checks=1

```

默认无额外运行期语义的 `interpreter-regression` 中, `optional_semantics_skip=15` 是解释器确认依赖局部默认值、空引用、负长度数组或除零等可选语义的样例。默认 `unchecked` 编译器不承诺这些行为, 因此不把它们纳入输出等价比较; 数组越界和函数末尾隐式 `return 0` 不在这个集合内, 默认仍要和解释器对齐。开启 `RUNTIME_CHECKS=1` 后, 局部默认初始化、空引用检查、负长度数组检查和除零检查等额外语义也全部与解释器一致。

`stress_runtime_skip=2` 对应老师新增的 `bigloop.fmj` 和 `linkedlist.fmj`。这两个程序仍然经过 `parser`、`semantic checker`、编译、链接和 `qemu smoke run`; 只是 `AST` 解释器逐语句执行过慢, 不把完整解释器输出比较放进常规回归。它们仍由 `all-mode-regression` 检查六种优化模式的一致性, 并由 `opt-benchmark` 用于性能测量。

因此 86 个 `reject` 均由解释器语义/语法检查确认, 33 个带 `EXPECT: FAIL` 的文件也都被编译器正确拒绝。2 个 `timeout` 是解释器和编译产物都超时的非终止程序, 作为行为一致处理。对定义良好的程序, 默认编译器与解释器完全一致; 对依赖可选运行期语义的程序, 开启选项后也与解释器一致。`all-mode-regression` 还会把每个文件在六种优化模式下的 `process return code`、`stdout`、`stderr` 和 `FMJ return value` 逐项比较, 确认优化不会改变可观察行为。

1.11.7 Fuzz 测试

对于包含外部输入的程序, `fuzz_regression.sh` 使用解释器作为 `oracle` 进行差分 fuzz。测试方式是:

1. 收集含 `getint`、`getch` 或 `getarray` 的 FMJ 程序。
2. 先用解释器和编译器分别检查语法/语义。
3. 对可接受程序，编译成 ARM 汇编，并把 `main` 改名为 `test_main`。
4. ARM 侧链接一个 `harness`，在同一个 `qemu` 进程中循环调用 `test_main()`。
5. `harness` 和解释器使用相同 `seed`、相同输入生成策略、相同输出 `hash`。
6. 每个程序执行 `ITERS=10000` 组随机输入。

fuzz 结果如下：

```
1 fuzz_pass=46 fuzz_fail=0 fuzz_skip=3 iterations=10000 kset="9" workdir=/tmp/↵
  ↵ final_fuzz_regression
```

其中 `fuzz_skip=3` 是三个 `spec-invalid` 的 HW2 原始测试：

```
1 HW2/test/fibonacci.fmj
2 HW2/test/test_comprehensive.fmj
3 HW2/test/test_io.fmj
```

这些文件包含 `int i = 0;` 或类似声明初始化，因此 `parser` 按 FDMJ2026 specification 拒绝。`fuzz` 脚本会先经过 `parser/semantic check`，只有被编译器接受的输入程序才进入 ARM `harness` 和解释器 `hash` 对照。

实际进入 ARM `harness` 和解释器 `hash` 对照的 46 个输入程序全部通过；被跳过的程序都是前置 `parser/semantic check` 已确认应当拒绝的程序。部分 `fuzz` 输出如下：

```
1 OK interpreter k=9 HW10/test/optloopivextra2.fmj 5bdac9544df208ea 10000
2 OK interpreter k=9 HW11/test/fibonacci.fmj a8d38cf26e53b73d 10000
3 OK interpreter k=9 HW9/test/opttest9.fmj 4110fa6c1bf841a3 10000
```

1.11.8 优化性能测试

老师新增的 `FINALREPORTtests/newtests.fmj` 中包含几个运行时间更长的程序。`make opt↵ ↵ -benchmark` 会选择 `bigarray.fmj`、`bigloop.fmj`、`deepnestedloops.fmj` 和 `bubblesort.fmj`，分别用六种优化模式编译、链接、运行，并比较每个模式的 `process return code`、`FMJ return value`、`stdout hash` 和 `stderr hash`。四个样例的 `bench_consistent` 均为 1，说明性能测量没有以改变可观察结果为代价。

实测环境同本次回归环境，`qemu` 单次运行，默认不开启额外运行期语义检查。耗时单位为毫秒：

文件	none	const	loop1	loop2	allloop	allopt
<code>bigarray↵ ↵ .fmj</code>	39.209	41.906	40.283	44.849	43.452	45.691

文件	none	const	loop1	loop2	allloop	allopt
bigloop↔ ↪ .fmj	70402.172	69552.142	67680.770	67881.041	67191.812	66977.106
deepnestedloops↔ ↪ .fmj	10.889	10.003	10.298	10.521	10.274	10.530
bubblesort↔ ↪ .fmj	10.114	10.523	9.721	9.648	9.756	9.670

同一批编译产物的 ARM 汇编行数如下：

文件	none	const	loop1	loop2	allloop	allopt
bigarray.fmj	131	122	144	149	159	171
bigloop.fmj	70	67	68	74	72	70
deepnestedloops.fmj	825	330	825	825	825	330
bubblesort.fmj	315	314	330	423	423	423

这些数字体现了两个现象。第一，优化不是单调减少汇编行数：LICM 会增加 preheader 计算，归纳变量强度削弱会增加循环内维护的派生变量，因此 bubblesort.fmj 在 loop2/allloop/allopt 下行数明显增加。第二，优化收益取决于样例结构：bigloop.fmj 的 loop1/loop2/allloop 比 none 快约 2% 到 3%，但 allopt 在这次单次 qemu 测量中接近 const；deepnestedloops.fmj 主要受常量传播影响，汇编行数从 825 降到 330，但 qemu 运行时间本身太短，单次计时只适合说明趋势，不能作为精确微基准。

1.11.9 运行时回归测试

为了快速检查后端链接和运行，另外保留了 runtime-regression，只跑 HW10 和 HW12 中更接近运行端到端的测试。结果如下：

```
1 total=24 pass=24 compile_fail=0 link_fail=0 run_fail=0 workdir=/tmp/↔
  ↪ final_runtime_regression
```

这些结果说明：final/test/all 中的 FMJ 文件都被编译器和解释器一致地接受或拒绝；all-mode-regression 覆盖的测试用例在六种优化模式下可观察结果一致；默认模式下，不依赖可选运行期语义的程序 stdout 和退出码完全一致，数组越界也与解释器的 exit(-1) 行为一致；开启运行期语义选项后，可选运行期错误和默认值语义也与解释器一致；含外部输入程序在 10000 组随机输入下与解释器 oracle 一致；所有不合法输入均被正确拒绝，没有编译器 crash。

1.12 实现中遇到的问题

编译器集成中的很多 bug 本质上不是某一条语句写错，而是某个阶段没有维持后续阶段依赖的不变量。例如控制流阶段假设每个 block 有合法 label，SSA 阶段假设 def/use 完整，后端假设地址模式满足 ARM 立即数限制。下面几个问题都说明了同一个原则：只要某个阶段输出的中间表示看似能打印、但没有满足形式化约束，错误往往会在更后面的优化或代码生成阶段才暴露。

1.12.1 空 label / 空 block

早期集成后，部分程序在中间 IR 中出现空入口 label 或空跳转目标。这个问题会一路传播到 Quad 和汇编，最终导致后端生成不可用控制流。修复方式是在 ast2tree 和控制流生成阶段保证每个 block 都有明确 label，所有条件跳转和无条件跳转都指向有效 label。compile ↵ ↵ -regression 中加入了专门扫描，当前 empty_label_hits=0。

这个问题在 Final 阶段比单次作业中更容易暴露，因为后端不再只消费少量手写 IR，而是要消费从全仓库 .fmj 程序自动生成的 IR。修复后，blocking()、SSA、LICM、IV/SR、调度和寄存器分配都能假设 block 入口和出口是有效 label。

1.12.2 Fallthrough 边上的 Phi Copy

SSA 中的 phi 函数不是实际机器指令，离开 SSA 形式时必须在每条前驱边上插入 copy。例如循环头 L102 中的 `i_2 = phi(i_0 from L116, i_1 from L114)`，需要在 `L116 -> L102` 和 `L114 ↵ ↵ -> L102` 两条边上分别插入 `i_2 = i_0` 与 `i_2 = i_1`。如果只处理显式 jump 或 cjump 边，而漏掉没有显式跳转的 fallthrough 边，寄存器分配会认为 phi 目标有颜色，但实际汇编中从未给对应物理寄存器赋值。

final/lib/instr/schedule.cc 中的 scheduler 已修复这个问题：当一个 block 的最后语句不是 return、jump、cjump 或 exit，且它有唯一后继 label 时，把这条边按 fallthrough successor 处理，先调用 appendPhiCopies()，再线性化后继 block 或跳转到已访问后继。贯穿例子中的 Acc^sum 正好覆盖了入口 block fallthrough 到循环头的情况，修复后所有优化模式输出一致。

1.12.3 可选运行期语义

原作业中的部分阶段默认测试较短，没有完整规定未显式初始化局部变量的行为。Final 默认不为普通局部 int 和局部数组引用补语义；解释器在 --runtime-status 中标记 default_int_zero ↵ ↵ 、default_ref_null 等原因，默认回归只比较不依赖这些语义的程序。函数 fallthrough return 则默认补 return 0，不算可选语义。--runtime-checks 开启后，编译器会补齐局部默认值等额外语义并与解释器逐项对齐。

这个设计的关键是把“编译器应当保证的定义良好程序行为”和“为了让解释器与 ARM 在未定义/未规定程序上完全一致而补的防御语义”分开。默认模式保持更接近课程作业代码；检查模式用于更强的差分测试。

1.12.4 后端立即数范围

ARM 的 load/store offset 立即数有范围限制。指令选择阶段若把任意常量都折叠进 $[\leftarrow \hookrightarrow \text{base}, \#imm]$ ，大数组或深字段偏移时会产生非法汇编。修复后只在合法范围内折叠，其他情况显式计算地址。这个修复对应 `selectInstr.cc` 中对 `isArmMemoryImmediate()` 的检查。

这个问题也影响优化后的程序。LICM 或 strength reduction 可能改变地址计算形态，如果后端无条件折叠 offset，优化正确的 Quad 仍可能生成非法 ARM。因此最终在指令选择阶段做最后一道 ARM 立即数合法性判断。

1.12.5 函数 prologue 位置

调度阶段曾把入口 label 放在 prologue 之前。这样普通顺序执行没有问题，但循环回边如果跳回入口 label，会重复压栈和调整 sp/fp。最终把 prologue 固定在线性化结果的入口 label 之前，保证回边不会重复执行函数入口代码。

贯穿例子中的 `Acc^sum` 有 while 回边。如果入口 label 和 prologue 顺序错误，某些回边路径可能重新执行函数入口栈帧建立，最终破坏返回地址或局部变量访问。修复后循环 label 只包围函数 body，不包含 prologue。

1.12.6 HW6-HW10 优化组合

HW6-HW10 在原作业中通常单独测试，但 Final 要求 `run-allloop` 和 `run-allopt` 把多个优化串起来。这里出现的主要问题是 pass 之间的数据结构约定：一个 pass 生成的新 Quad 必须保留完整 def/use、block 入口出口、program last label/temp、function last label/temp。否则下游 pass 可能仍能打印 Quad，但 SSA、flow 或 register allocation 会在更后面失败。

最终 `fmjcc/main.cc` 每运行一个 pass 后都会调用 `refreshQuadExtents()`，并重新计算 flow 信息，而不是复用上一个 pass 的 flow。这样每个 pass 都以当前 Quad 真实结构为准，减少跨 pass 的隐藏状态。

1.13 结论

Final 工程把 HW2-HW12 中分散实现的 parser、semantic checker、IR 翻译、Quad、SSA、优化、指令选择、调度、寄存器分配和 ARM 汇编生成整合为一个可构建、可运行、可测试的 FMJ 编译器。`make compile` 会输出所有中间表示，`make run*` 可以分别验证无优化、单项优化、循环优化组合和全部优化下的 ARM 程序。

从贯穿例子可以看到，同一个源程序中的数组、对象、方法调用和循环会在每个阶段被逐步降低：AST 保存语法结构，语义表补充类型和名字解析，IR+ 表达运行时布局，Quad 显式化 def/use，SSA 用 phi 表达循环中的多路径定义，优化 pass 在 SSA 上改写程序，后端再把 Quad 选择成 ARM 指令并完成寄存器分配。回归测试和 fuzz 结果显示，定义良好的 FMJ 程序在解释器和编译后 ARM 运行之间保持一致，不合法输入也能被稳定拒绝。

1.14 参考资料

1. HW2/docs/FDMJ2026Specification.md, FMJ 语法和语言边界。
2. HW2-HW12 已完成代码与报告。
3. PARSING/ 中的 flex/bison parser 源码。
4. HW12/Makefile 和 HW12/vendor/libsysy, ARM 链接和 qemu 运行方式。